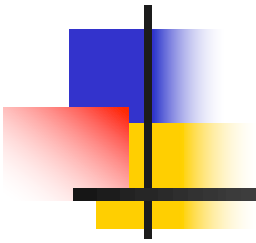
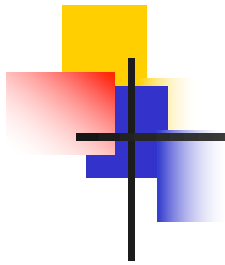


7.4 Caso de uso “comprar”

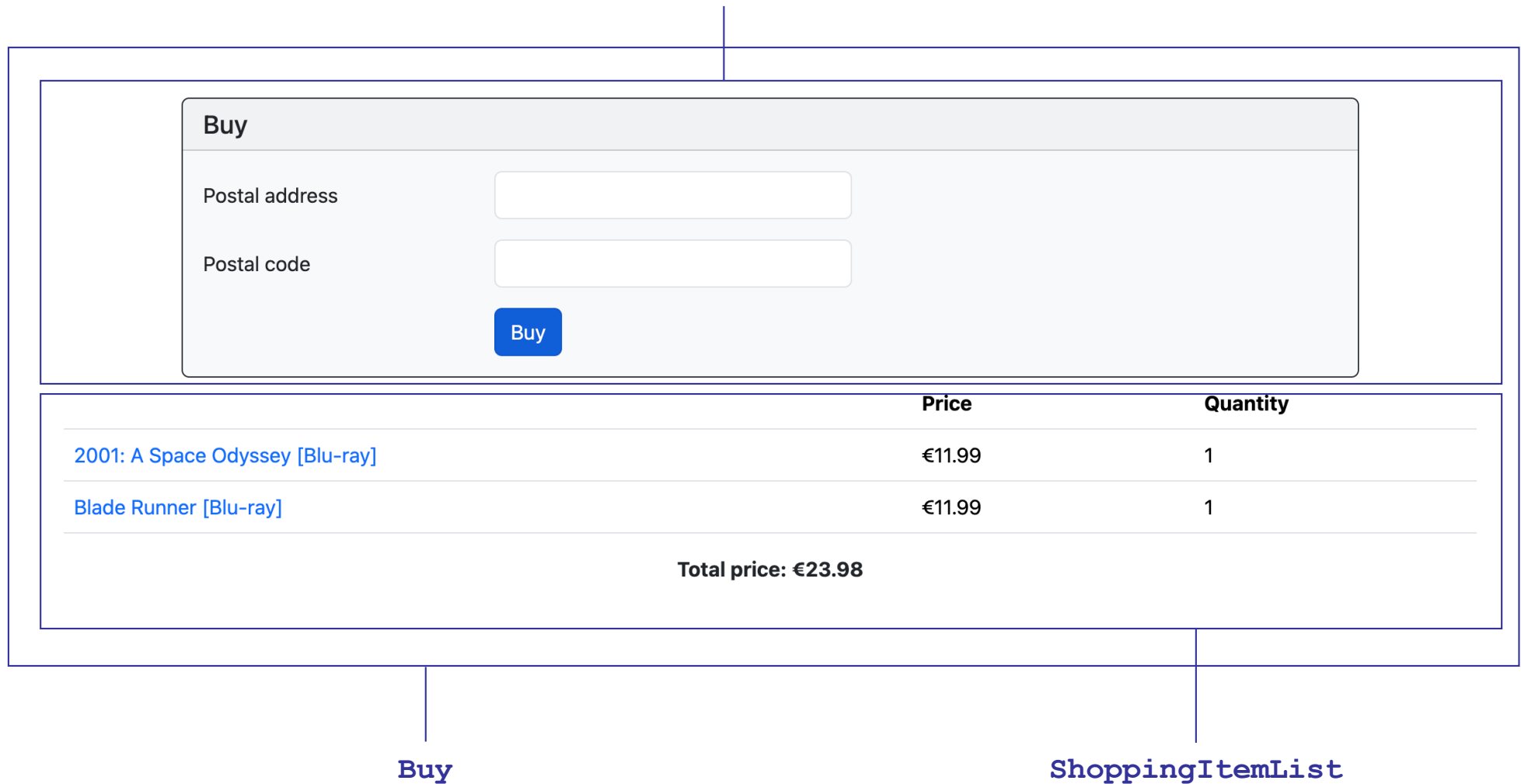


- Componentes
 - Validaciones del lado cliente
 - Gestión de errores devueltos por el backend
- Acciones
- Reductor



Componentes I

BuyForm



Componente Buy

```
...  
import ShoppingItemList from './ShoppingItemList';  
import BuyForm from './BuyForm';  
import * as selectors from '../selectors';
```

```
const Buy = () => {
```

Estado global

```
const cart = useSelector(selectors.getShoppingCart);
```

```
if (cart.items.length === 0) {  
  return null;  
}
```

```
return (  
  <>
```

```
    <BuyForm shoppingCartId={cart.id}/>
```

```
    <ShoppingItemList list={cart}/>
```

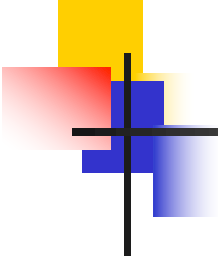
```
  </>
```

```
);
```

```
}
```

```
export default Buy;
```

Este componente se reutiliza aquí, en detalle de pedido y en mostrar carrito (ver código fuente)



Componente BuyForm

```
...  
import {Errors} from '../../common';  
import * as actions from '../actions';  
  
const BuyForm = ({shoppingCartId}) => {
```

```
    const dispatch = useDispatch();  
    const navigate = useNavigate();
```

Controlled component: una variable de estado por cada campo de entrada del formulario

Estado local

```
    const [postalAddress, setPostalAddress] = useState('');  
    const [postalCode, setPostalCode] = useState('');
```

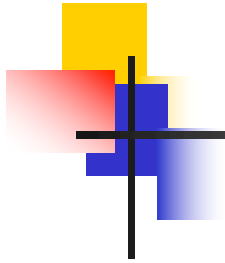
```
    const [backendErrors, setBackendErrors] = useState(null);
```

Errores devueltos por el backend (objeto con propiedad `globalError` o `fieldErrors`)

```
    const [formValidated, setFormValidated] = useState(false);
```

React Bootstrap (se explica más adelante)

```
    let form;
```



Componente BuyForm

```
const handleSubmit = async event => {  
  
  event.preventDefault();  
  
  if (form.checkValidity()) {  
  
    const response = await backend.shoppingService.buy(  
      shoppingCartId, postalAddress, postalCode);  
  
    if (response.ok) {  
      dispatch(actions.buyCompleted(response.payload));  
      navigate('/shopping/purchase-completed');  
    } else {  
      setBackendErrors(response.payload);  
    }  
  
    } else {  
      setBackendErrors(null);  
      setFormValidated(true);  
    }  
  }  
}
```

Si respuesta exitosa => se pasa a la Pantalla PurchaseCompleted

Si respuesta devuelve errores => se continúa en esta pantalla y se muestran los errores

Componente BuyForm

- Visualiza los errores devueltos por el backend (ver código fuente en el módulo `common`)
- No muestra nada si el valor de `errors` es `null`

```
return (  
  <div className="...">  
    <Errors errors={backendErrors}  
      onClose={() => setBackendErrors(null)} />  
    <Card className="...">  
      <Card.Header as="h5">  
        <FormattedMessage id="...title" />  
      </Card.Header>  
    </Card>  
  </div>  
)
```

Errors

- Postal code: size must be between 1 and 20

BuyForm

Card

Card.Header

Card.Body

Buy

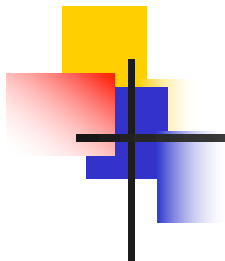
Postal address: Rúa del Percebe, 13

Postal code: 123456789012345678901

Buy

React Bootstrap

- `Card` y `Card.Body` generan el tag `div` añadiendo estilos CSS
- `Card.Header` genera, en este caso, el tag `h5` añadiendo estilos CSS



Componente BuyForm

```
<Card.Body>
```

```
  <Form ref={node => form = node}>
```

```
    validated={formValidated} noValidate
```

```
    onSubmit={(e) => handleSubmit(e)}>
```

```
    <Form.Group ... controlId="postalAddress">
```

Nombre

```
      <Form.Label ...>
```

```
        <FormattedMessage id="...postalAddress"/>
```

```
      </Form.Label>
```

```
      <Col ...>
```

```
        <Form.Control type="text"
```

```
          value={postalAddress}
```

```
          onChange={e =>
```

```
            setPostalAddress(e.target.value) }
```

```
          autoFocus
```

```
          required/>
```

Entrada

Errores
de lado
cliente

```
        <Form.Control.Feedback type="invalid">
```

```
          <FormattedMessage id='...required' />
```

```
        </Form.Control.Feedback>
```

```
      </Col>
```

```
    </Form.Group>
```

Campo de entrada
postalAddress

Componente BuyForm

Campo de entrada
postalCode

```
<Form.Group ...>
  ...
</Form.Group>
```

```
<Form.Group ...>
  <Col ...>
    <Button type="submit">
      <FormattedMessage id="...buy"/>
    </Button>
  </Col>
</Form.Group>
```

```
</Form>
```

```
</Card.Body>
```

```
</Card>
```

```
</div>
```

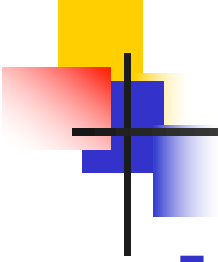
```
);
```

```
}
```

```
...
```

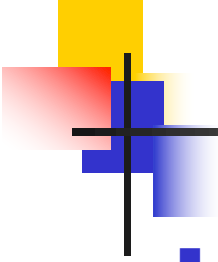
React Bootstrap

- `Form`, `Form.Control` y `Button` se comentaron en el apartado 7.2
- El uso concreto que se hace de `Form.Group` y `Col` (generan el tag `div` añadiendo estilos CSS) permite que los campos del formulario se dispongan de forma horizontal
- `Form.Label` genera el tag `label` añadiendo estilos CSS
- `Form.Control.Feedback` genera el tag `div` añadiendo estilos CSS para mostrar los errores de las validaciones del lado cliente



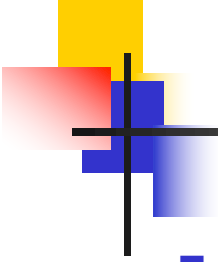
Validaciones del lado cliente

- Para implementar las validaciones del lado cliente (navegador), el caso de estudio usa el soporte de validación de formularios de HTML5
 - https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation
 - Soportado por las últimas versiones de los navegadores más populares (<https://caniuse.com/form-validation>)
 - Alternativas: librerías de validación o soluciones ad-hoc
- El soporte de validación en HTML5 se puede usar de dos formas
 - Declarativamente
 - Programáticamente



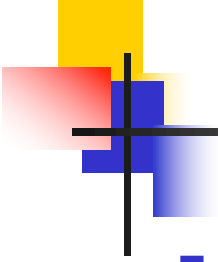
Validaciones del lado cliente

- Enfoque declarativo
 - Cada campo puede especificar las restricciones que se deben cumplir
 - Algunas son implícitas al tipo de campo
 - `<input type="email" id="email">`
 - Se crea automáticamente una restricción para validar que el valor introducido concuerda con la sintaxis de una dirección de correo electrónico
 - Otras se pueden añadir a algunos campos para establecer restricciones adicionales
 - `<input type="text" id="name" required minlength="6" maxlength="20">`
 - `<input type="number" id="age" min="18" max="100">`
 - Demo: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation#using_built-in_form_validation
 - Más Información: https://developer.mozilla.org/en-US/docs/Web/HTML/Guides/Constraint_validation



Validaciones del lado cliente

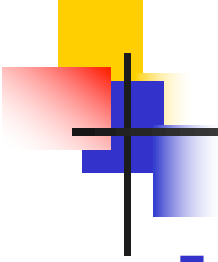
- Enfoque declarativo
 - Ventaja: no hay que programar
 - Desventajas
 - No hay una forma estándar de adaptar el look-n-feel de los mensajes de error y el propio contenido de los mensajes
 - Los mensajes se muestran de acuerdo al locale del navegador, y no de acuerdo, al locale de la aplicación



Validaciones del lado cliente

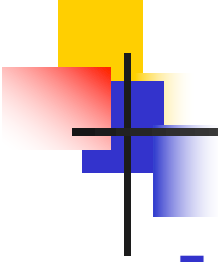
- Enfoque programático

- Al igual que en el enfoque declarativo, el desarrollador puede continuar especificando restricciones sobre los campos de entrada
- Pero a diferencia del enfoque declarativo, el navegador no ejecutará las validaciones automáticamente
- El navegador proporciona una API estándar para ejecutar las validaciones y permite añadir mensajes de error con contenido propio
- Más información: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation#using_built-in_form_validation



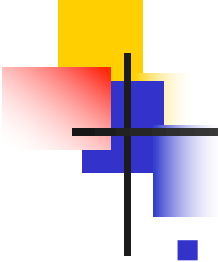
Validaciones del lado cliente

- El caso de estudio utiliza el soporte de validación de formularios de HTML5 de forma programática
- Y también el soporte de Bootstrap, mediante la librería React Bootstrap, para implementar las validaciones del lado cliente con un look-n-feel atractivo
 - <https://react-bootstrap.github.io/docs/forms/validation>



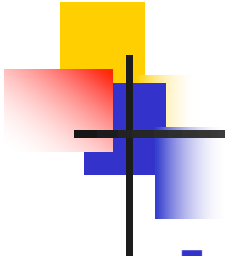
Validaciones del lado cliente

- Por sencillez, se usan pocas restricciones
 - Las implícitas al tipo de campo (e.g. `email` y `number`) y algunas adicionales (`required` y `min`)
- **De todas formas, por seguridad, el backend tiene que ejecutar todas las validaciones** (las que se hagan en el frontend y las que no)
 - Y los posibles errores se mostrarán con el componente **Errors**



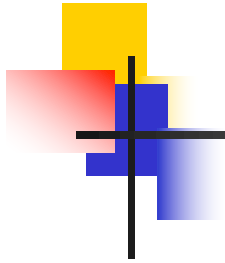
Validaciones del lado cliente

- En **Form** (React Bootstrap)
 - Se usa el atributo **noValidate**: deshabilita las validaciones automáticas del navegador (enfoque programático)
 - Se usa el atributo **validated** con la variable de estado **formValidated**
 - Si **validated** es **true**, se muestran las validaciones del lado cliente
- Flujo
 - El usuario hace clic en el botón del formulario
 - Se invoca a **handleSubmit**
 - Se ejecutan las validaciones del formulario invocando a **checkValidity** sobre el nodo del formulario
 - Si alguna se incumple, se establece el valor de la variable de estado **formValidated** a **true**
 - El componente **BuyForm** se vuelve a renderizar
 - El formulario indica qué campos pasan las restricciones y cuáles no
 - Cada campo usa **Form.Control.Feedback** (React Bootstrap) para mostrar su posible mensaje de error

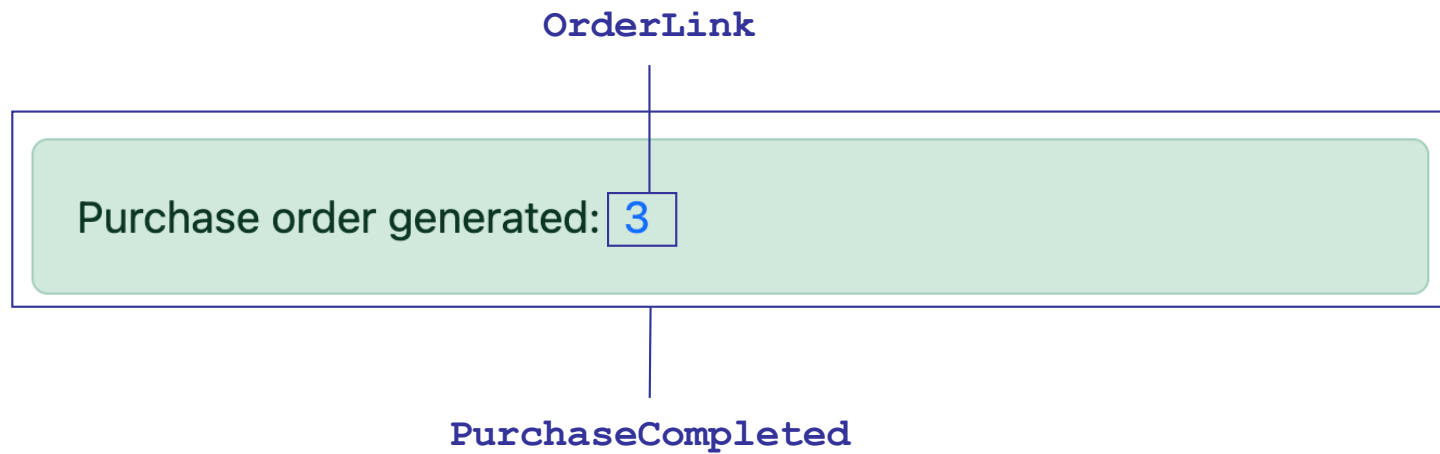


Gestión de errores devueltos por el backend

- Los errores devueltos por el backend en la respuesta se mantienen en el **estado local** del componente, dado que no son de interés para otros componentes
- Flujo
 - El usuario hace clic en el botón del formulario
 - Se invoca a `handleSubmit` y se envía la petición al backend
 - Si la respuesta devuelve errores, se guardan en la variable de estado `backendErrors`
 - El componente `BuyForm` se vuelve a renderizar
 - Como parte de la renderización, `Errors` imprime los errores del objeto `backendErrors`
 - Si el usuario hace clic sobre la "x" de `Errors`, se establece `backendErrors` a `null`
 - El componente `BuyForm` se vuelve a renderizar
 - Como parte de la renderización, esta vez `Errors` no imprime nada (el valor de `errors` es `null`)



Componentes II



Componente PurchaseCompleted

```
...
import * as selectors from '../selectors';
import OrderLink from './OrderLink';

const PurchaseCompleted = () => {

  const orderId = useSelector(selectors.getLastOrderId)

  if (!orderId) {
    return null;
  }

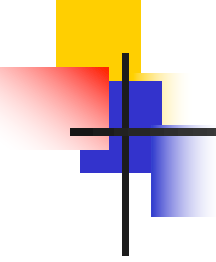
  return (
    <Alert variant="success">
      <FormattedMessage id="...purchaseOrderGenerated"/>:
      &nbsp;
      <OrderLink id={orderId}/>
    </Alert>
  );
}

export default PurchaseCompleted;
```

Estado global: se establece desde
BuyForm y se recupera desde aquí

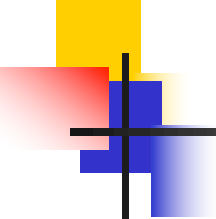
Ver código fuente (módulo shopping). En este caso,
genera:

```
<Link to={` /shopping/order-details/${id}`} >
  {id}
</Link>
```



Acciones

```
export const buyCompleted = (orderId) => ({  
  type: actionTypes.BUY_COMPLETED,  
  orderId  
});
```



Reducer

```
import {combineReducers} from 'redux';
..
import * as actionTypes from './actionTypes';

const initialState = {
  shoppingCart: null,
  lastOrderId: null,
  orderSearch: null
};

const shoppingCart = ...

const lastOrderId = (state = initialState.lastOrderId, action) => {

  switch (action.type) {
    case actionTypes.BUY_COMPLETED:
      return action.orderId;
    default:
      return state;
  }
}
```



Reducer

```
const orderSearch = ...
```

```
const reducer = combineReducers({  
  shoppingCart,  
  lastOrderId,  
  orderSearch,  
});
```

```
export default reducer;
```